

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über [GitHub](#) [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF11a:00:Hintergrund zur Shellprogrammierung

Shell-Programming

- LNX: History:
 - Ideen:
 - * 1965 im Rahmen einer Konferenz ‘Fall Joint Computer Conference’ Aufsätze über ein neu zu erstellendes Betriebssystem ‘Multics’
 - * Multics wurde bis 1980 entwickelt, war aber nie erfolgreich und ist letztlich gescheitert
 - * Seine Ideen wurden nach Unix übernommen.
 - Ursprung:
 - * 1969 zogen sich ‘AT&T/Bell Labs’ aus dem Projekt ‘Multics’ zurück.
 - * Intern in den Bell Labs gingen die Arbeiten für ein Mehrbenutzersystem weiter
 - * Mangels Ressourcen mussten die Entwickler eine ungenutzte alte schmalbrüstige Maschine PDP-7 ‘reaktivieren’. Dafür gab es nicht mal ein eigenes Entwicklungssystem.
 - * Dafür schrieben sich Ken Thompson, Dennis Ritchie etc. auf einem anderen System namens GECOS zuerst:
 - ein Dateisystem + ein primitives Prozessmanagement + eine Reihe kleinerer Programme, die das System benutzbar machen sollten:
 - den Editor ed,
 - kleinere Dateiverwaltungsprogramme,
 - ein einfacher Kommandozeileninterpreter, später Unix-Shell sh genannt,
 - und zwar bis das System schließlich ausreichend ausgestattet war, um ohne den GECOS-Umweg direkt auf der PDP-7 weiterentwickelt zu werden.
 - Hierin findet die UNIX-Philosophie seinen Ursprung: Unix/Linux folgt der Maxime, dass kleine Tools nur ein Zweck erfüllen sollen und ineinandergreifend aufrufbar sein sollen. Das geschieht über Pipes, die Texte an das nachfolgende Programm als Input weiterreichen
 - * In AT&T/Bell Lab gewinnt das Projekt an Renomé. Es gibt eine neue Maschine PDP-10

- * 1970 entsteht der Name „Unics“: UNiplexed Information and Computing Service, der später verkürzt wird zu Unix.
 - * 1971 wird eine interpretierte Programmiersprache für die PDP-7 namens B entwickelt mit eigenem Compiler
 - * 1972 Neuerstellung von Unix in dieser Sprache, jetzt C genannt.
 - * 1973 Die Portierung von Unics abgeschlossen und erhielt den Namen „UNIX V4“, also Version 4 von UNIX.
- Vertrieb:
- * AT&T war 1956 per Consent Decree verboten worden, neue Märkte wie des Computermarktes zu betreten.
 - * Deshalb stellt sie die Unix-Software in der 1975 aktuellen Version 6 für lediglich den Preis der Datenträger verschiedenen Universitäten zur Verfügung, mitsamt dem vollständigen Quellcode.
 - * Die Universitäten, insbesondere die Universität von Kalifornien in Berkeley, entwickelten das System weiter.
 - * Zu dieser Zeit wurde alle Programme in universitär frei ausgetauscht. U.a. in/mit der Berkeley Software Distribution (BSD).
 - * Die Universitäten entwickelten das System weiter. Z.B. stammt die TCP/IP Kommunikation aus dieser Phase.
 - * Zu diesem Zeitpunkt war Unix ein Dateisystem, ein Prozessmanager (Kernel), ein Editor, ein Compiler und eine Shell + die vielen Tools und Verbesserungen der Universitäten
 - * Die Grund-Philosophie ‘viele kleine Tools, die sich per Pipe ihre Ergebnisse zuspielen’, blieb erhalten!
 - * 1979 durch AT&T die letzte UNIX-Version mit freiem Quellcode, UNIX V7, veröffentlicht.
 - * Danach begann Proprietarisierung des Codes.
 - * 1981: MS-DOS (Microsoft Disk Operating System) auf den Markt. App 1977 Beginn des Personalcomputers.
 - * 1983: Start GNU Projekt, initiiert v. Richard M. Stallman: Ziel ein vollständiges Betriebssystem auf der Basis von Freier Software zu schaffen.
 - Erste Arbeit: Programmierung GCC (steht heute GNU Compiler Collection)
 - Danach eigener Editor emacs.
 - Übersetzung der bisher freien Tools (BSD etc.) mit GSD.
 - * 1989: Erfindung der GNU License.
 - Zu diesem Zeitpunkt war das Betriebssystem GNU (als Alternative zum jetzt proprietären UNIX) eine Sammlung der alten freien Tools (deshalb die alten Lizenzen BSD / MIT) + die GNU-Tools (GNU make etc.) **PLUS** ein alter Unix-Kernel.
 - Für die Eigenständigkeit fehlte ein eigener GNU Kernel. GNU arbeitete deshalb an

HURD, kam damit aber nicht voran.

- * (vgl. https://de.wikipedia.org/wiki/Geschichte_von_Unix bzw. https://de.wikipedia.org/wiki/Geschichte_von_Linux)
- * 1991 beginnt Linus Torvalds auf der Basis von Minix (einem universitären Studienprojekt) erst einen 'Terminalemulator', der sich zum eigenen Kernel entwickelte,
- * Linus lieferte dem GNU-Projekt, was ihm fehlte: einen Kernel. Zuerst prototypierte er GCC und bash auf sein System. Damit war die Lücke geschlossen
- * 1992: erste offiziell auf CD erhältliche Linux-Distribution war Yggdrasil Linux, entwickelt von Adam J. Richters
- * 1992: Gründung von SuSE
- * 1993 veröffentlichte Patrick Volkerding die Distribution Slackware,
- * 1993, ungefähr einen Monat nach der Veröffentlichung von Slackware, wurde das Debian-Projekt ins Leben gerufen
- * 1994 SuSE 1.0 als vorkompilierte Slackware Distro auf CD/DVD
- * 1994 RHL = RedHat Linux Distro (Ab 2003 zu RHEL Enterprise Linux)
- * 1996 erste stabile Debian version
- * 2004 das auf Debian basierende Ubuntu von Canonical
- * Für solche Distros entstand die Kurzform LINUX, obwohl sie viele BSD- / MIT-Software enthielt und exzessiv auf dem GCC und anderer GNU-Software beruhte.
- * Stallman hat erfolglos durchzudrücken versucht, dass von GNU/Linux gesprochen werden sollte, weil die meisten darin enthaltenen Tools unter der GPL/LGPL oder anderen freien Lizenzen veröffentlicht worden sei und weil LINUX eigentlich nur der (fehlende) Kernel des GNU-Projektes sei. Angenommen wurde das nicht.

- Spirit/Philosophie:

- Viele kleine Tools, die NUR das tun, wofür sie gedacht sind.
- Diese lesen Daten von STDIN ein (oder von Parametern / Infos) und geben das Ergebnis nach STDOUT aus.
- Sie werden über Pipes verknüpft (Pipes sind Sockets ;-))
- `ls /bin/cat` zeigt den Pfad zum Programm cat an.
- `cat` selbst nimmt 1 - n Pfade zu Dateien als Parameter, concateniert sie und gibt das Ergebnis aus
- `cat ./shue1er1.md ./shue1er2.md` ist `schue1er.md`
- `cat ./shue1er1.md ./shue1er2.md > schue1er.a`
- `cat ./shue1er[0-9].* > schue1er.b`
- Vorführen `tee`, `find` ...

- Tools:

- `ls /bin/ | grep '^[A-Za-z]'` (alle tools mit Buchstaben begonnen)
 - `ls /bin/ | grep '^[A-Za-z][A-Za-z]'`
 - `ls /bin/ | grep '^[A-Za-z][A-Za-z][A-Za-z]'`
 - `ls /bin/ | grep -E '^[A-Za-z]{3}'`
 - `ls /bin/ | grep -E '^[A-Za-z]{4}'` Und wie nur pPogrammnamen mit genau 4 Buchstaben?
 - `ls /bin/ | grep -E '^[A-Za-z]{4}$'`
 - Beweis mit `ls /bin/ | grep -E '^[A-Za-z]{4}$' | grep 'grep'` success
 - Beweis mit `ls /bin/ | grep -E '^[A-Za-z]{3}$' | grep 'grep'` fail
 - Alternative `find / | grep 'grep'` sehr lang
 - Besser? `find /bin | grep 'grep'` liefert nur bin
 - `ls -la /bin` liefert erklärung: bin ein link auf `/usr/bin`
 - Besser? `find /usr/bin | grep 'grep'` liefert zuviel
 - Besser! `find /usr/bin -name "grep"`
 - Datenaufbereitung mit sort:
 - * `cat schueler[0-9]* | sort | uniq | grep -v ^$`
 - * Umwandlung in eine CSV-Datei, nach Nachnamen sortiert
 - * `cat schueler[0-9]* | sed "s/ /,/g" | grep -v ^$ | awk -F " ", '{print $2 ", " $1 }' | sort | uniq`
 - **Gemeinsam:** Die Tools werden in einer shell ausgeführt
- LNX: kennt eine Reihe verschiedener shells. Alle dienen dazu, commandline-Programme aufzurufen und deren Ausgabe anzuzeigen
 - `cat schueler.md | sort | uniq`
 - sh (Bourne Shell) erste Unix-Shell überhaupt. Entwickelt bei AT&T. Nachteile:
 - * keine eingebauten Kommandos
 - * keine Commandhistory + recall
 - * begrenzte IO-Funktionalität
 - bash = GNU Bourne Again Shell (Wortspiel)
 - * gleicht die Mängel der sh aus. Heute Standard im User-bereich
 - csh = C shell der Universität California, bietet History und aliases
 - ksh = korn shell = entwickelt bei AT&T, will ebenso Nachteile der sh ausgleichen
 - zsh = Z Shell will sh und csh ultimativ verbessern
 - dash = Debian Almquist Shell = kleine, schnellere POSIX-konforme 'Schwester' der Bash.
 - Grundsätzlicher Unterschied: Syntax für Shell-Skripte und eingebaute Befehle.
 - vgl.

- * <https://www.digitalocean.com/community/tutorials/different-types-of-shells-in-linux>

*

Hinweis: Unter Unix/Linux ist die Bash heute Standard. Das geht so weit, das dort `bin/sh` ein Link auf `bin/bash` oder `bin/dash` ist. Wer bash-Skripte schreibt, kann sie in der Regel von der bash oder dash ausführen lassen. (Ubuntu/Debian verlinkt sh auf dash) [`$(which bash)` -> `bash`, `$(which sh)` -> `dash`] Deshalb kann für alle die Shebang-Zeile `#!/bin/sh` verwendet werden. Eine Shebangzeile legt fest, von welchem Programm ein Script ausgeführt werden soll, wenn es als ausführbar markiert worden ist (755)

- Shell-Programm

s. Datei `schueler.sh`

- WIN:

- kam ursprünglich nur mit der Command-Shell einher
 - * Suche nach `cmd` oder `command`. Eingabeaufforderung aufrufen.
 - * Ruft einfache Programme auf > Hinweis: Die Shell und die Eingabeaufforderung unterscheiden sich in Syntax und Semantik massiv, auch wenn die Themen eng verwandt sind: beide haben ein Befehl `date`. Unter LNX liefert `date --help` eine lange Liste von Möglichkeiten, unter Windows kommt die Rückmeldung, das Datum konnte nicht gesetzt werden. Unter LNX liefert `ifconfig` die Wertebelegung der Netzwerkinterfaces. Unter WIN heißt das Kommando `ipconfig`
- Im Jahr 2003 hat Windows (Unix-)Admins eingeladen, um ihre Wünsche an eine Admin-Shell zu erfahren. Ergebnis: So wie unter UNIX/Linux.
- Daraufhin entstand die Powershell als eine auf den .NET-Framework aufsetzende Shell, auch mit eingebauten Befehlen.
 - * Version 1.0 im Jahr 2006
 - * Version 2.0 im Jahr 2008
 - * Version 3.0 im Jahr 2012
 - * Version 4.0 im Jahr 2013
 - * Version 5.0 im Jahr 2015 zusammen mit Windows 10 (für ältere in Version 5.1 nachgereicht)
 - * Version 6.0 im Jahr 2018 ist eine neue Open-Source-Version [Powershell-Core] (MIT) gehostet auf GitHub, die auch für LNX erhältlich ist.
 - * Version 7.+x ist die aktuelle, 5.1 kriegt nur noch updates, 6 wird nicht weiterentwickelt, Basis ist jetzt die Version 7.x plattformübergreifend. > Hinweis: Durch die Architektur im .Net-Framework enthält die Powershell 7 eine Fülle von eingebauten Befehlen. Unter

LNx gibt es weiter das Prinzip der vielen kleinen eingebauten Tools. Die Powershell versucht Ihre Funktionalität damit zu simulieren. Das geht aber nur begrenzt. Deshalb ist sie unter LNx nur bedingt ein Ersatz. > Außerdem: Die Philosophie von Unix ist es, schlanker Server haben zu können. Es wird nur das installiert, was nötig ist. Bei der Powershell müssen die .NET-Teile mit installiert sein. Auch auf dem Remote-Rechner, auf den man sich zur Remotewartung einloggt.

- Ermitteln der eigenen Version:
 - * Powershell starten
 - * `$psversiontable` eingeben
- Installation Version 7.4:
 - * via Windows Store ohne Admin-rechte. nach Powershell 7 suchen.
- PSH spricht von Commandlets, wenn es seine eingebauten Befehle meint
- Commandlets sind nach dem Muster `${Verb}-${Substantiv} [Parameter]` aufgebaut
- Beispiel: `Get-Service`: liefert alle lokal laufenden Prozesse
- Als Parameter kann man hier `i*` eingeben und erhält alle Prozesse die mit i beginnen.
- Es gibt darüber hinaus Aliasse: `Get-Alias | Where-Object {$_.definition -eq "Get-Process"}`
- `ps` ergibt dasselbe Ergebnis wie `Get-Process`

Hinweis: die Suchabkürzungen ähneln den Regular Expression, weichen aber in einigen Punkten ab `ps i*` meint: alles, was mit i beginnt. Das müsste als Regex `i.*` ausgedrückt werden. [Exkurs Regex] Hinweis: so nähern sich die beiden System an. An den Parametern ist der Unterschied aber leicht zu erkennen: LNx: `ps aux` liefert alle laufenden Prozesse, `ps` nur die des Users. WIN: PSH versteht die Parameter nicht.

Aufgabe: `schueler1.md` und `schueler2.md` mit Windowsmitteln in der Powershell in einer CSV-Datei nach Nachname sortiert umformen.